Algorithmic Trading Visualization Workbench

A human-centered web application for building, testing, exploring, and comparing algorithmic trading strategies through a visual interface.

What this project does

The workbench helps a user move through one connected workflow:

Create Strategy → Configure Parameters → Run Backtest → View Results → Explore Variations → Compare Strategies

The main idea is simple: users can build strategies using visual blocks or code, run backtests, and understand the results without switching between many screens or remembering previous configurations.

Problem statement

Algorithmic trading interfaces can be difficult to use because users have to work with strategy rules, parameters, backtests, charts, and performance information at the same time.

This project focuses on making that process easier to understand by providing a visual workbench for strategy construction, backtesting, parameter exploration, comparison, and result interpretation.

Objectives

- Make strategy creation easier through code blocks.
- Make backtesting simple to configure and run.
- Show backtest progress and results clearly.
- Help users explore how parameter changes affect results.
- Help users compare different strategy runs with enough context.
- Reduce avoidable user errors and confusion.
- Improve accessibility and overall usability based on user testing.

Main features

1. Dashboard

Provides the main entry point to the workbench and the project workflow.

2. Strategy Builder

Allows users to build a strategy using available indicators and rule controls instead of starting from raw code.

3. Parameters

Allows the user to configure strategy values and the historical period used for the backtest.

4. Backtesting

Runs the selected strategy on historical data and shows the system state while the backtest is running.

5. Backtest Results

Shows performance information, trade information, and visual results so that users can understand what happened.

6. Parameter Exploration

Lets users change parameters, run variations, and inspect how results change.

7. Strategy Comparison

Lets users compare multiple runs and understand the differences between them.

Tech stack

Frontend

- **React** – main UI framework.
- **JavaScript / JSX** – application logic and UI components.
- **Vite** – frontend development and build tool.
- **React Router** – page navigation.
- **CSS** – layout, styling, and visual design.

Backend and backtesting

- **C#** – backend layer used to connect the interface with the backtesting workflow.
- **LEAN** – backtesting engine used by the project workflow.
- **Historical market data** – used as the input for backtests.

Development and version control

- **Git** – version control.
- **GitHub** – remote repository and collaboration.
- **VS Code / similar IDE** – development environment.

High-level architecture

```
Historical Market Data
        ↓
Visual Strategy Builder
        ↓
Parameter Configuration
        ↓
Backtesting Backend
        ↓
LEAN Backtesting Engine
        ↓
Performance & Trade Results
        ↓
Interactive Visualization
        ↓
Parameter Exploration
        ↓
Strategy Comparison
        ↓
User
```

The architecture is intended to keep the user workflow connected from strategy creation to final comparison.

User flow

Getting started

Prerequisites

Install the following before running the project:

- Node.js and npm
- Git
- The required C#/.NET development environment for the backend
- LEAN and the project backtesting workspace configured as required by the repository

Clone the repository

```
git clone https://github.com/rishavrai563/Algorithmic-Trading-Workbench.git
cd Algorithmic-Trading-Workbench
```

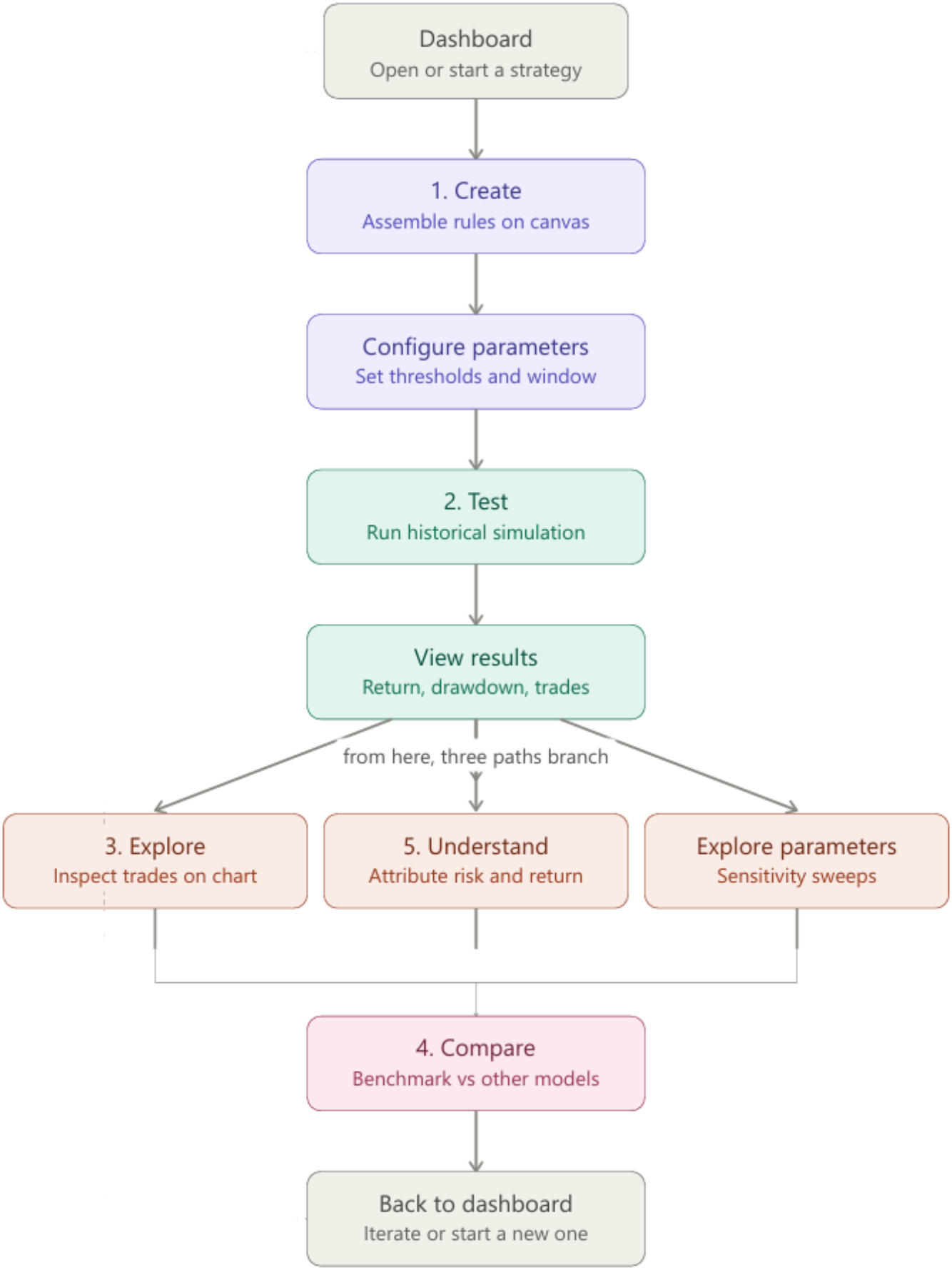


Figure 1: User Flow diagram showing the path from Dashboard through Strategy Builder, Parameters, Backtest, Results, and then splitting into Explore, Understand, and Compare

Backend

Download the LEAN image once before the first backtest:

```
docker pull quantconnect/lean:latest
```

Run Locally

Start the backend in one terminal:

```
cd backend
dotnet restore
dotnet run --urls http://localhost:5000
```

Start the frontend in another terminal:

```
cd frontend
npm install
npm run dev
```

Build

Build the frontend:

```
cd frontend
npm run build
```

Build the backend:

```
cd backend
dotnet build
```

Team Members

Name	Entry No.	GitHub
Ravikant Sharma	2024AIB1013	https://github.com/thyravikant
Rishav Kumar	2024AIB1014	https://github.com/rishavrai563

Course: **AI511 – HCI & Visualization**
Institute: **Indian Institute of Technology Ropar**

API Documentation

Overview

This document outlines the REST API endpoints provided by the ASP.NET Core backend. The API is divided into two main controllers: `StrategiesController` for managing strategy state and history, and `BacktestController` for executing and monitoring LEAN backtests. All responses are formatted in JSON.

1. Strategies Controller

Base Route: `/api/strategies`
Manages CRUD operations for user strategies and stores historical backtest records.

`GET /api/strategies`

- **Description:** Retrieves a list of all saved strategies.
- **Payload/Query:** None

`GET /api/strategies/{id}`

- **Description:** Retrieves a single strategy profile by its unique ID.
- **Payload/Query:** URL Parameter: `id` (string)

`POST /api/strategies`

- **Description:** Creates a new blank strategy in the database.
- **Payload/Query:** JSON body (StrategyProfile)

`PUT /api/strategies/{id}`

- **Description:** Updates and saves an existing strategy (e.g., modified code or parameters).
- **Payload/Query:** URL Parameter: `id`, JSON body (StrategyProfile)

`DELETE /api/strategies/{id}`

- **Description:** Deletes a strategy from the database.
- **Payload/Query:** URL Parameter: `id` (string)

Backtest History

`GET /api/strategies/history`

- **Description:** Retrieves all historical backtest records across all strategies.
- **Payload/Query:** None

`GET /api/strategies/{id}/history`

- **Description:** Retrieves the historical backtest variations run for a specific strategy.
- **Payload/Query:** URL Parameter: `id` (string)

`POST /api/strategies/{id}/history`

- **Description:** Records the performance metrics and parameters of a newly completed backtest run.
- **Payload/Query:** URL Parameter: `id`, JSON body (BacktestRecord)

2. Backtest Controller

Base Route: `/api/backtest`
Handles the orchestration of the Dockerized LEAN execution engine and external asset fetching.

`POST /api/backtest`

- **Description:** Submits a strategy configuration and Python code to run in the LEAN engine. Spawns an asynchronous job.
- **Payload/Query:** JSON body (BacktestRequest)

`GET /api/backtest/{id}/status`

- **Description:** Polls the status of a running backtest job. Returns progress percentages and final results when complete.
- **Payload/Query:** URL Parameter: `id` (string)

`GET /api/backtest/status`

- **Description:** System health check. Returns Docker availability and LEAN image status.
- **Payload/Query:** None

`GET /api/backtest/assets`

- **Description:** Retrieves a list of statically supported fallback assets for the frontend.
- **Payload/Query:** None

`GET /api/backtest/assets/search`

- **Description:** Proxy endpoint that queries the Yahoo Finance API to provide universal ticker autocomplete suggestions.
- **Payload/Query:** Query Parameter: `q` (string)